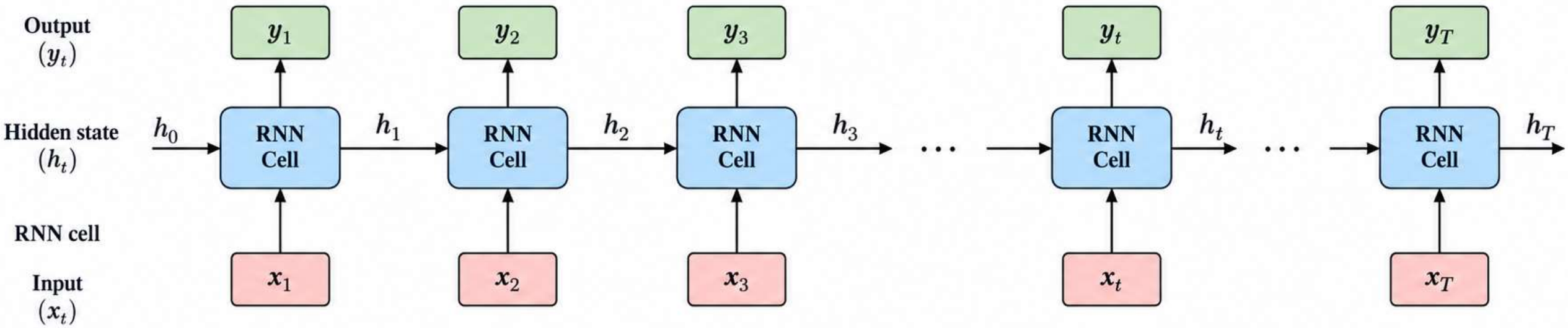
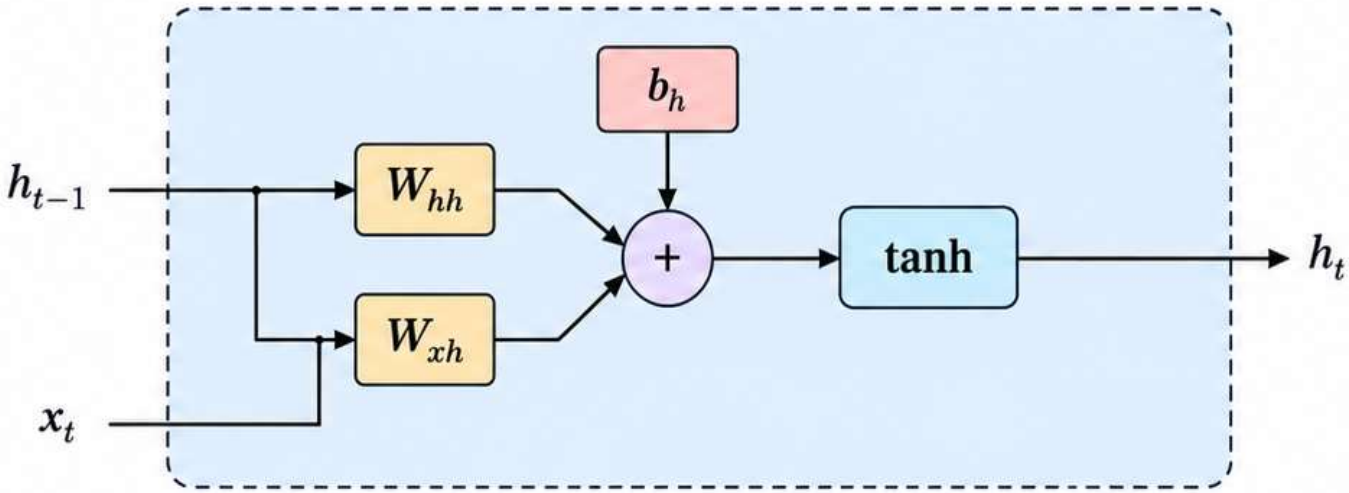
RNN Architecture (Unrolled View)



RNN Cell (Internal Structure)



RNN Equations

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right)$$

$$y_t = W_{hy}h_t + b_y \quad (\text{or} \quad y_t = \text{Softmax}(W_{hy}h_t + b_y))$$

x_t	: input at time step t (dimension: d_x)
h_t	: hidden state at time step t (dimension: d_h)
y_t	: output at time step t (dimension: d_y)
W_{xh}	: input-to-hidden weight matrix ($d_h \times d_x$)
W_{hh}	: hidden-to-hidden weight matrix ($d_h \times d_h$)
W_{hy}	: hidden-to-output weight matrix ($d_y \times d_h$)
b_h	: hidden bias (d_h)
b_y	: output bias (d_y)
tanh	: activation function

Recurrent Neural Network (RNN)

406

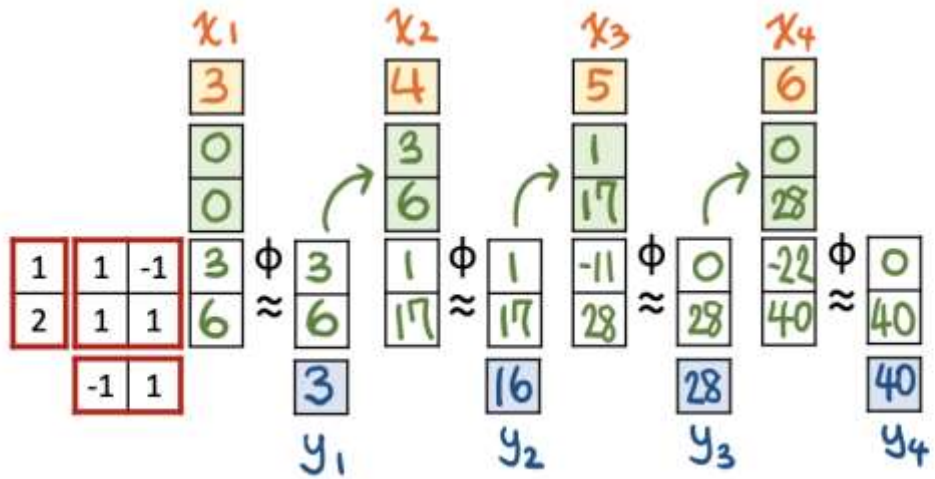
Input Sequence X [3 | 4 | 5 | 6]

Parameters A [[1, -1], [1, 1]] B [[1], [2]] C [-1, 1]

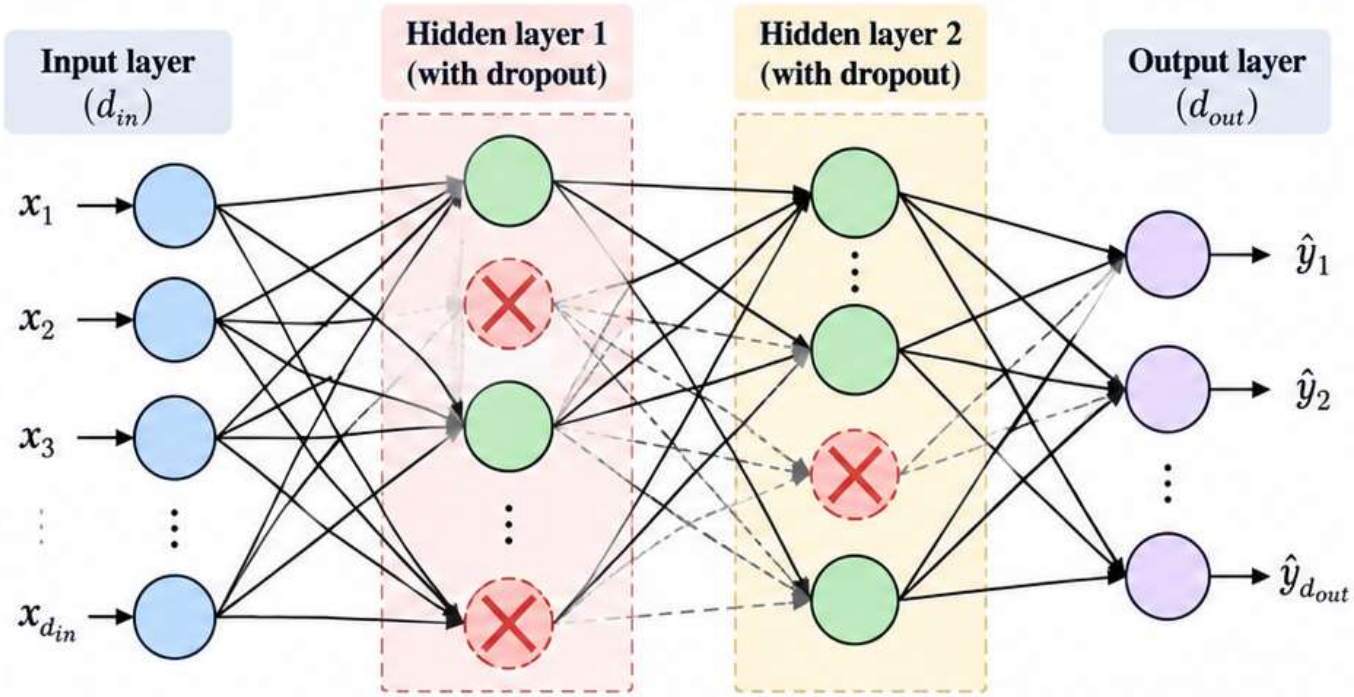
Activation Function ϕ: ReLU

Hidden States H_0 [[0], [0]]

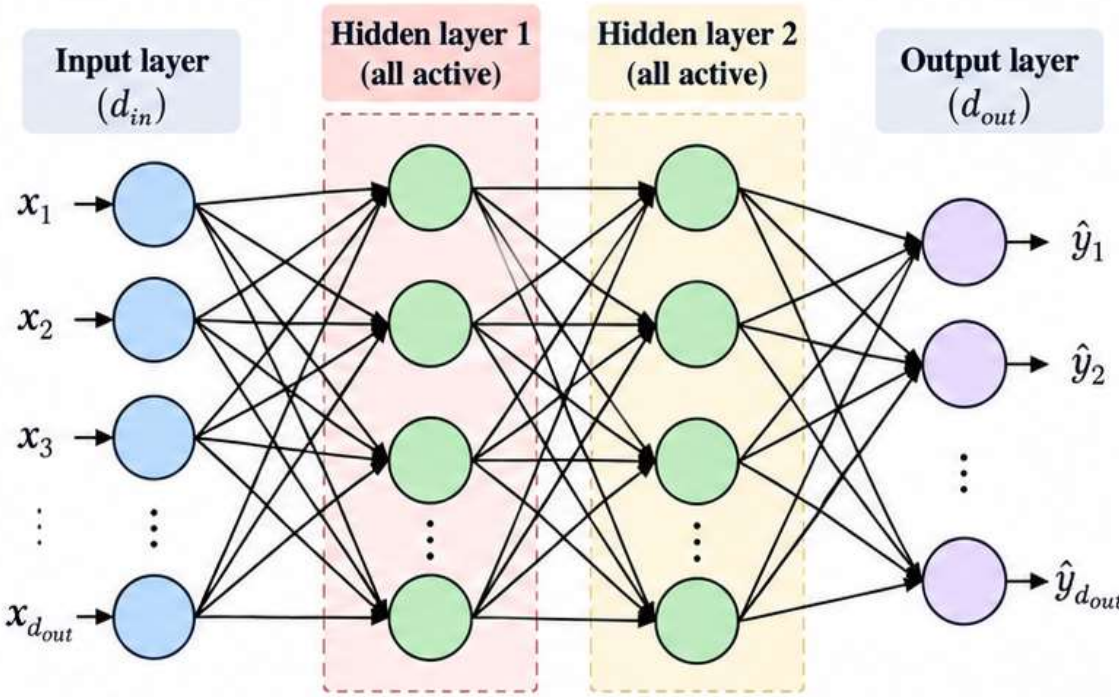
Output Sequence Y [| | |]



Dropout Architecture (During Training)



During Inference (No Dropout)



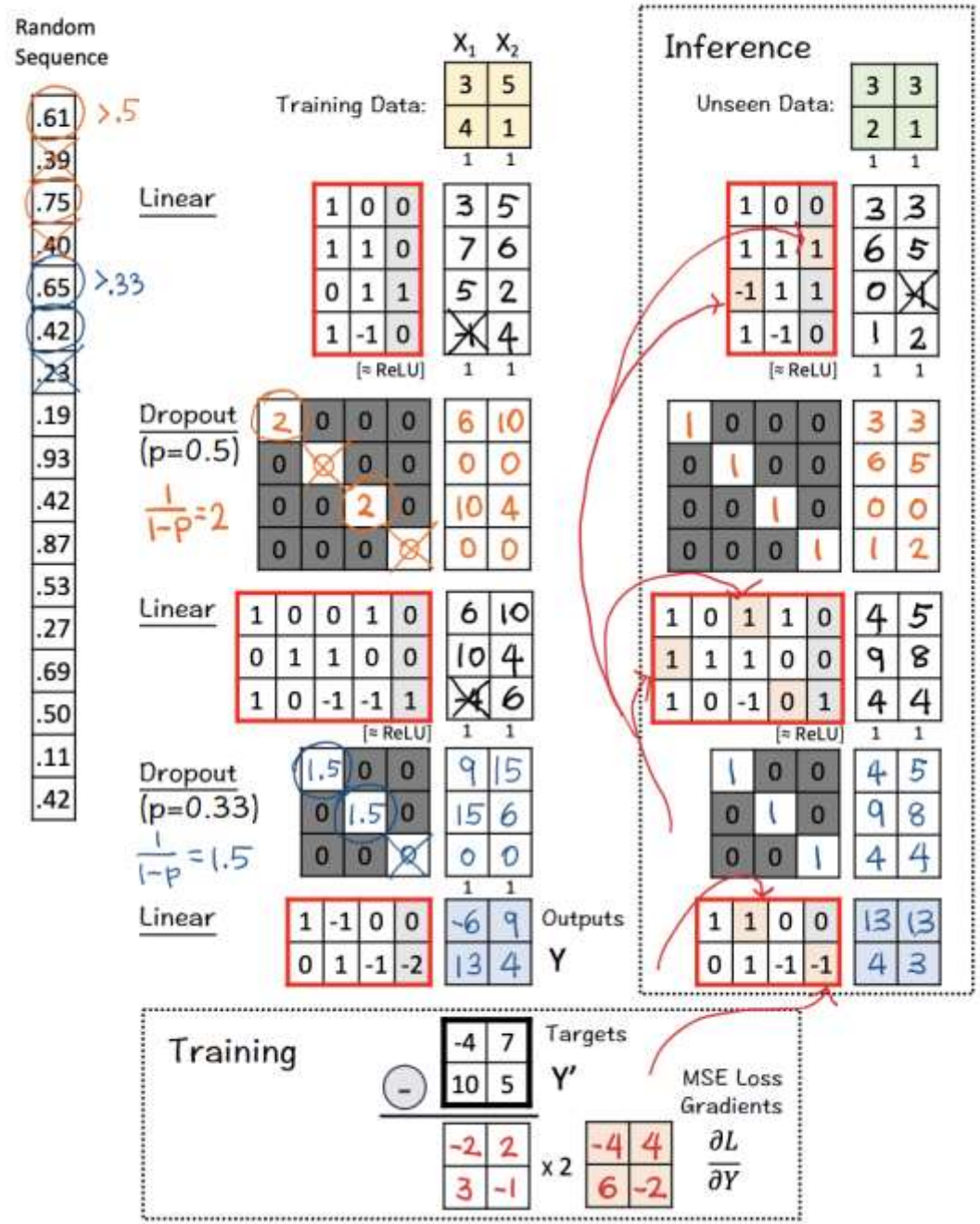
Legend

- Active neuron (kept during forward pass)
- Dropped neuron (output set to 0)
- Active connection
- Connection ignored due to dropout

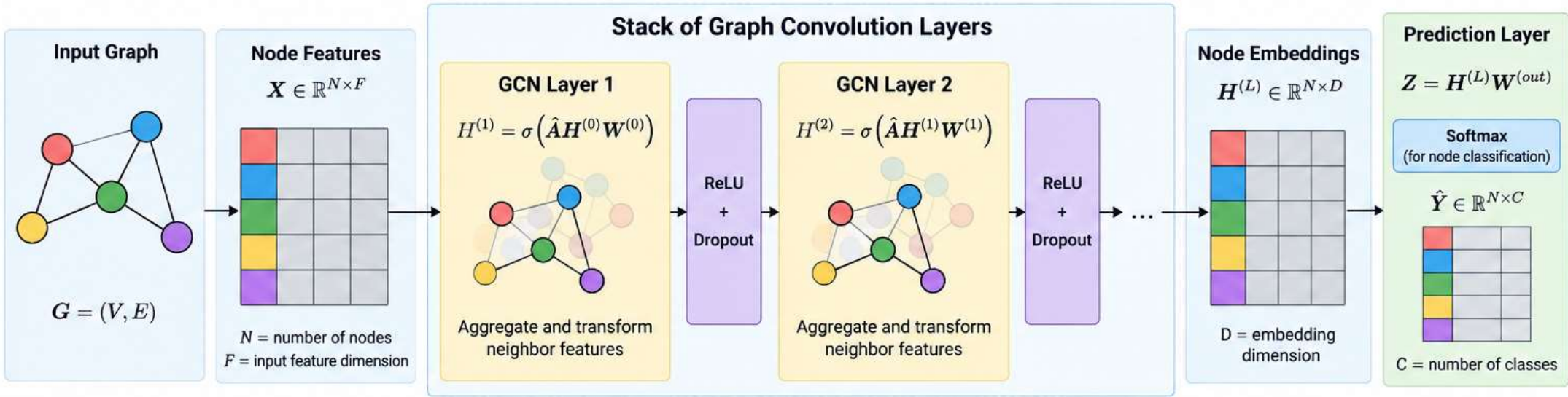
Key Points

- During training, each neuron is kept with probability p (e.g., $p = 0.5$) and dropped with probability $1 - p$.
- Dropped neurons have their output set to 0 and their outgoing connections are ignored.
- During inference (testing), no neurons are dropped. All neurons are used, typically with weights scaled by p (or using inverted dropout during training).
- Dropout helps reduce overfitting by preventing co-adaptation of neurons.

Dropout


Random Sequence
.61
.39
.75
.40
.65
.42
.23
.19
.93
.42
.87
.53
.27
.69
.50
.11
.42
>.5
>.33
Training Data:
X1 X2
3 5
4 1
Linear
1 0 0
1 1 0
0 1 1
1 -1 0
[≈ ReLU]
Dropout (p=0.5)
1/1-p=2
Linear
1 0 0 1 0
0 1 1 0 0
1 0 -1 -1 1
[≈ ReLU]
Dropout (p=0.33)
1/1-p=1.5
Linear
1 -1 0 0
0 1 -1 -2
Outputs
Y
Inference
Unseen Data:
3 3
2 1
[≈ ReLU]
[≈ ReLU]
Training
Targets
Y'
MSE Loss Gradients
x 2
∂L/∂Y

Graph Convolutional Network (GCN) Architecture



1. Graph Convolution Operation

$$H^{(l+1)} = \sigma\left(\hat{A}H^{(l)}W^{(l)}\right)$$

where:

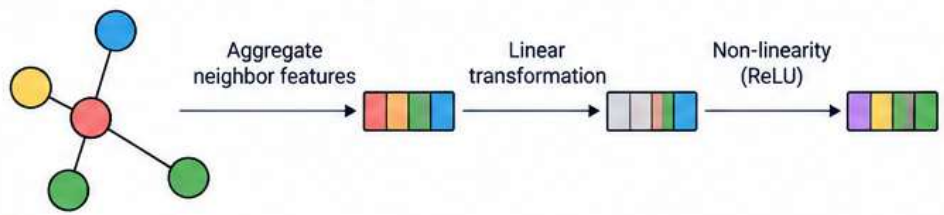
- $H^{(l)} \in \mathbb{R}^{N \times F_l}$: node features at layer l
- $W^{(l)} \in \mathbb{R}^{F_l \times F_{l+1}}$: learnable weight matrix
- $\sigma(\cdot)$: non-linear activation (e.g., ReLU)
- $\hat{A}$: normalized adjacency matrix

$$\hat{A} = \tilde{D}^{-1/2}\hat{A}\tilde{D}^{-1/2}$$

$$\tilde{A} = A + I$$

- A : adjacency matrix $(N \times N)$
- I : identity matrix (self-loops)
- $\tilde{D}$: degree matrix of $\tilde{A}$

2. Layer-wise Propagation (Intuition)

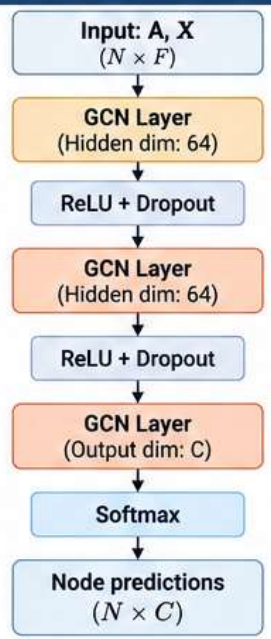


Node *i* and its neighbors

What happens in each layer?

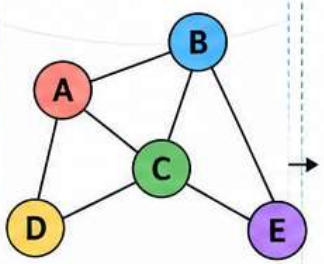
1. Each node aggregates (averages/sums) features from its neighbors (including itself).
2. The aggregated features are linearly transformed using a learnable weight matrix.
3. A non-linear activation (e.g., ReLU) is applied.
4. Stacking multiple layers allows information to propagate to higher-order neighbors (e.g., 2-hop, 3-hop, ...).

3. Full GCN Architecture (Example)



Graph Convolutional Network (GCN) Architecture

1. Graph Data



2. Adjacency Matrix A

	A	B	C	D	E
A	2	0	1	0	1
B	1	1	0	0	0
C	0	0	-1	1	1
D	0	3	0	1	0
E	1	0	0	1	0

3. Node Feature Matrix X

	A	B	C	D	E
A	1	1	0	0	0
B	0	1	0	-1	0
C	1	0	0	1	-1
D	0	1	1	0	-2
E	1	-1	0	0	0

$X \in \mathbb{R}^{5\times 5}$

4. Graph Convolution Layer 1

$$H^{(1)} = \sigma\left(\hat{A}XW^{(0)}\right)$$

Neighbor Aggregation $\hat{A}X$ → Linear Transformation $(\hat{A}X)W^{(0)}$ → Non-linearity **ReLU** $\sigma(\cdot)$

5. Node Embeddings (after Layer 1)

$H^{(1)} \in \mathbb{R}^{5\times F_1}$

	A	B	C	D	E
A	3	1	0	1	
B	0	1	0	0	
C	1	2	0	0	
D	4	1	5	2	
E	1	2	1	1	

→ ...

6. Graph Convolution Layer 2

$$H^{(2)} = \sigma\left(\hat{A}H^{(1)}W^{(1)}\right)$$

Neighbor Aggregation $\hat{A}H^{(1)}$ → Linear Transformation $(\hat{A}H^{(1)})W^{(1)}$ → Non-linearity **ReLU** $\sigma(\cdot)$

7. Node Embeddings (after Layer 2)

$H^{(2)} \in \mathbb{R}^{5\times F^{\prime}}$

	A	B	C	D	E
A	1	3	1	0	
B	2	3	0	1	
C	4	1	5	2	
D	2	6	2	6	
E	5	0	3	0	

8. Graph Convolution Layer L

$$H^{(L)} = \sigma\left(\hat{A}H^{(L-1)}W^{(L-1)}\right)$$

Neighbor Aggregation $\hat{A}H^{(L-1)}$ → Linear Transformation $(\hat{A}H^{(L-1)})W^{(L-1)}$ → Non-linearity **ReLU** $\sigma(\cdot)$

9. Final Node Embeddings

$H^{(L)} \in \mathbb{R}^{5\times D}$

	A	B	C	D	E
A	2	6	2	6	
B	5	0	5	0	
C	3	10	4	8	
D	0	4	0	4	
E	2	1	2	1	

10. Fully Connected Layer (for Graph-level or Node-level Prediction)

Node embeddings $H^{(L)}$ $(5 \times D)$

	A	B	C	E
A	2	6	2	6
B	5	0	5	0
C	3	10	4	8
D	0	4	0	4
E	2	1	2	1

Linear Layer

$$Z = H^{(L)}W^{(out)}$$

$(5 \times C)$

$Z \in \mathbb{R}^{5\times C}$

	A	B	C
A	0	4	0
B	3	3	2
C	2	3	1
D	0	6	6
E	3	8	2

Softmax

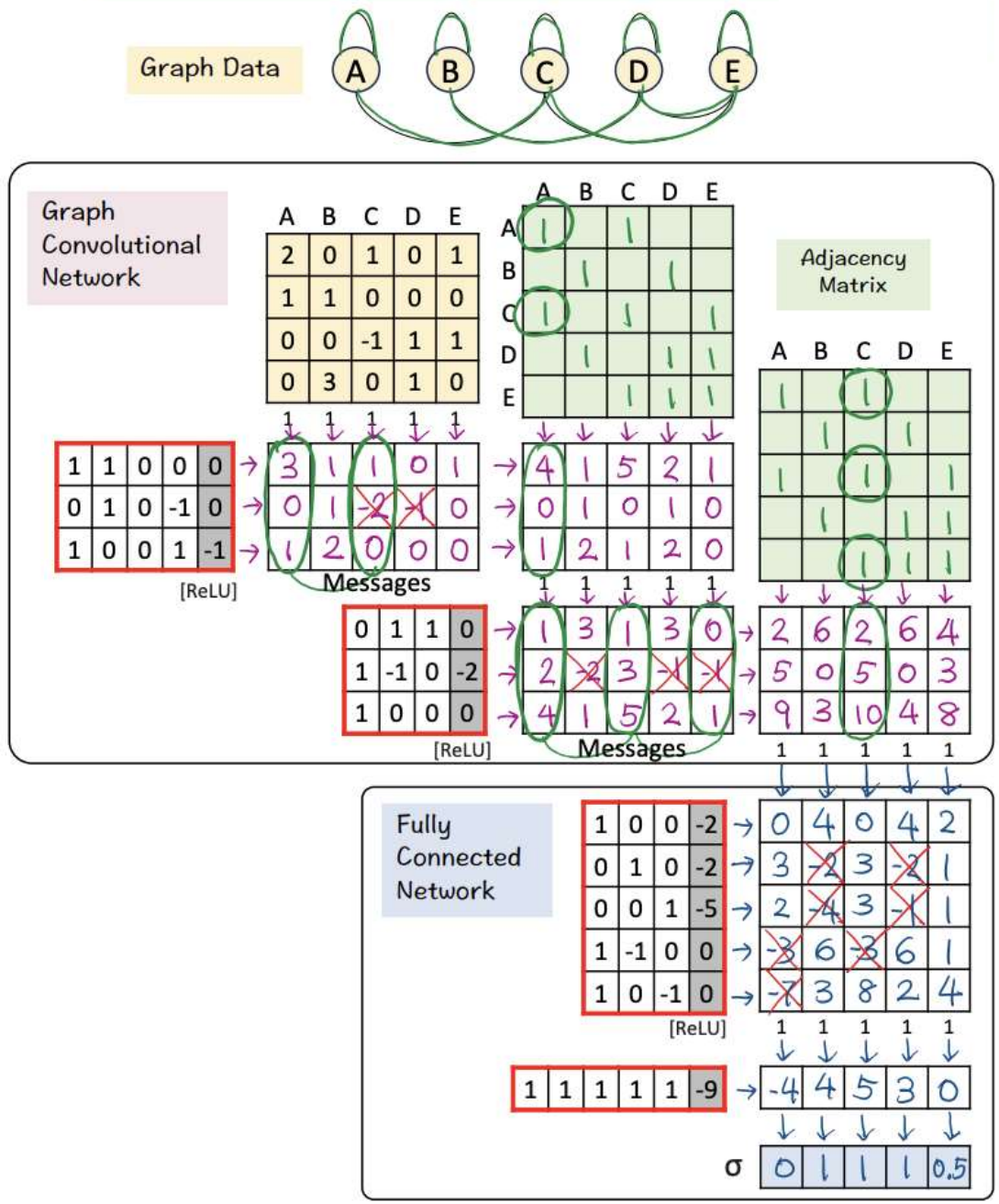
(for node classification)

$\hat{Y} = \text{softmax}(Z)$

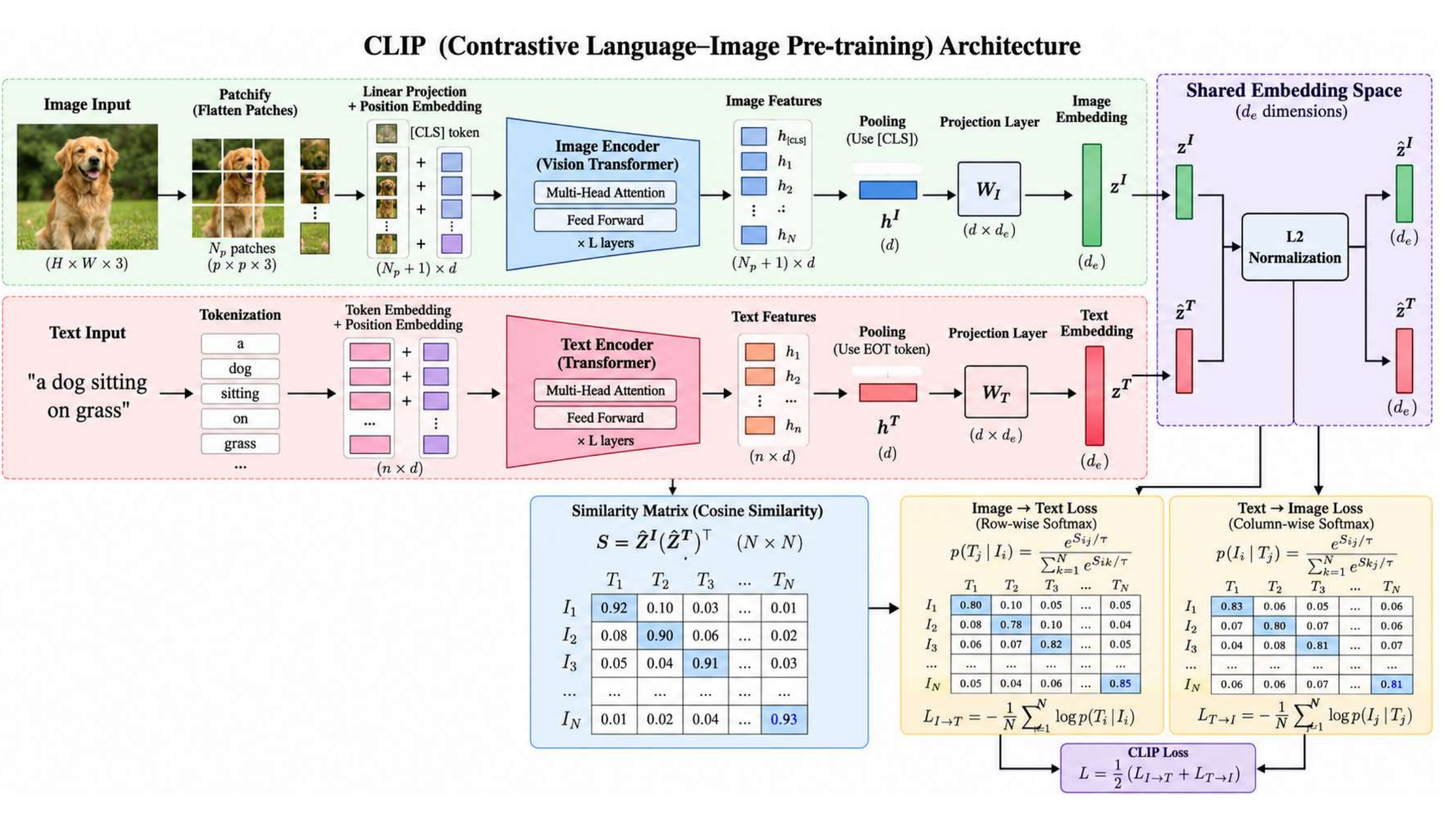
Node predictions $\hat{Y} \in \mathbb{R}^{5\times C}$

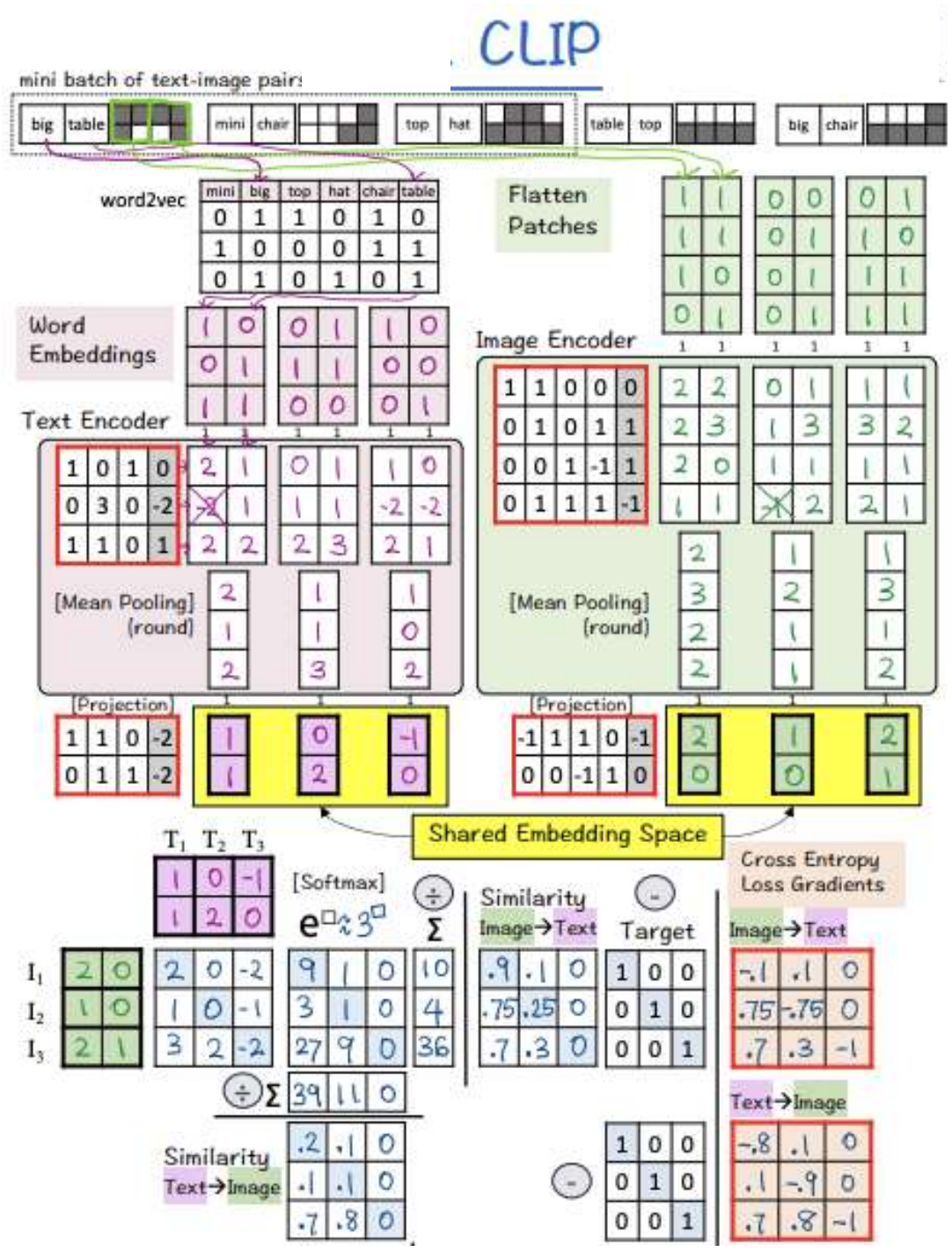
	A	B	C
A	0.1	0.7	0.2
B	0.4	0.4	0.2
C	0.2	0.6	0.2
D	0.1	0.7	0.2
E	0.3	0.6	0.1

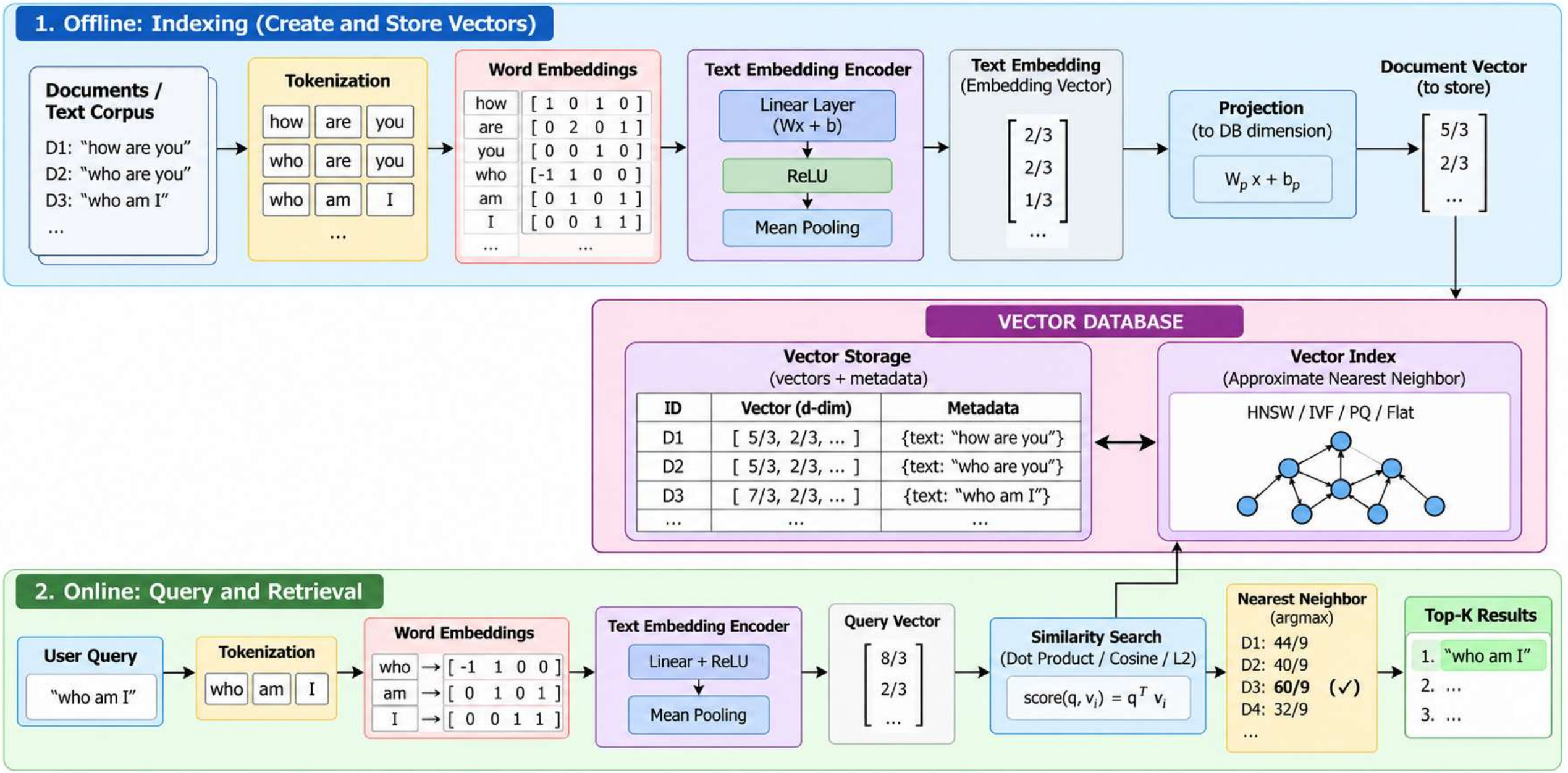
Graph Convolutional Network



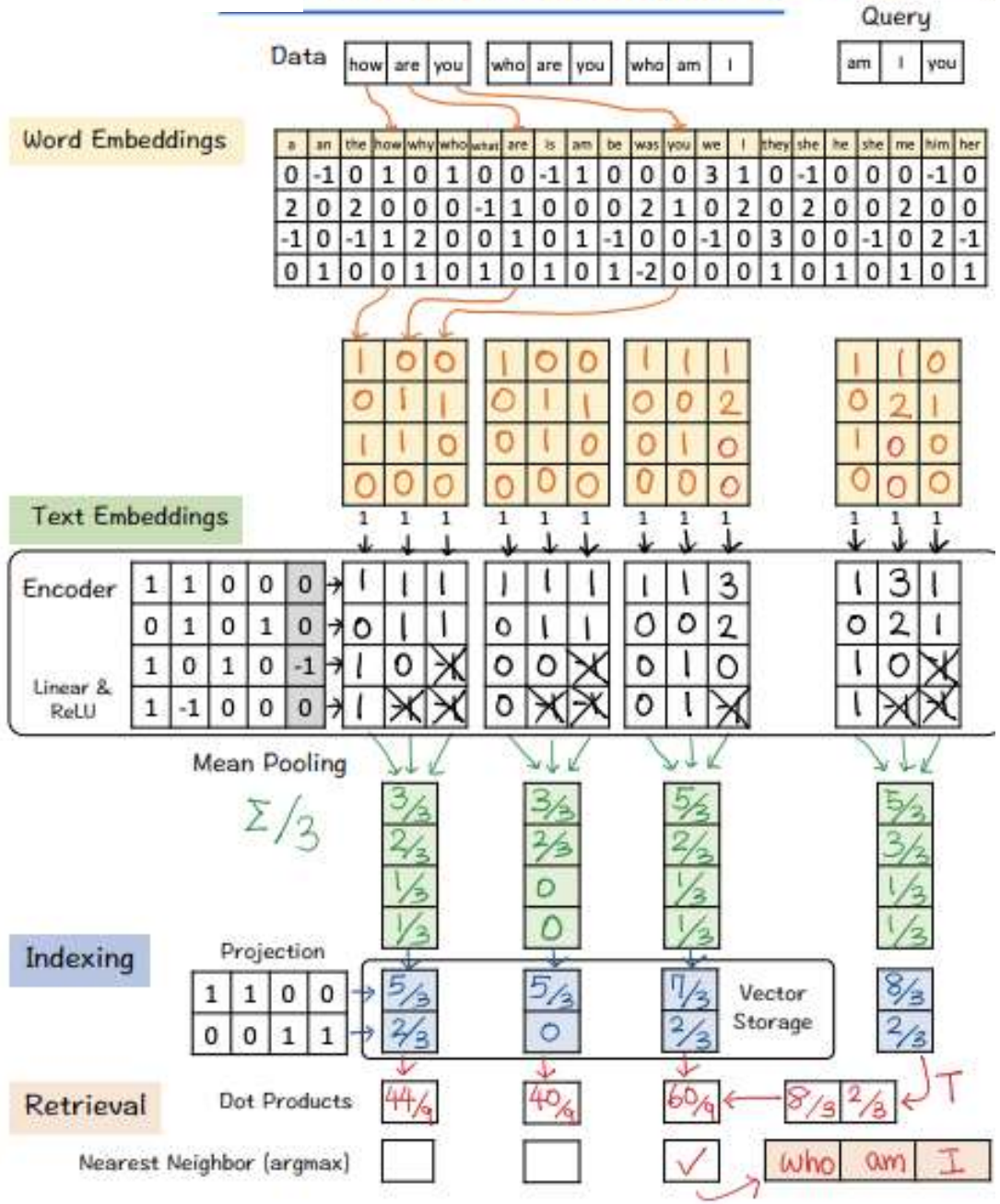
CLIP (Contrastive Language–Image Pre-training) Architecture



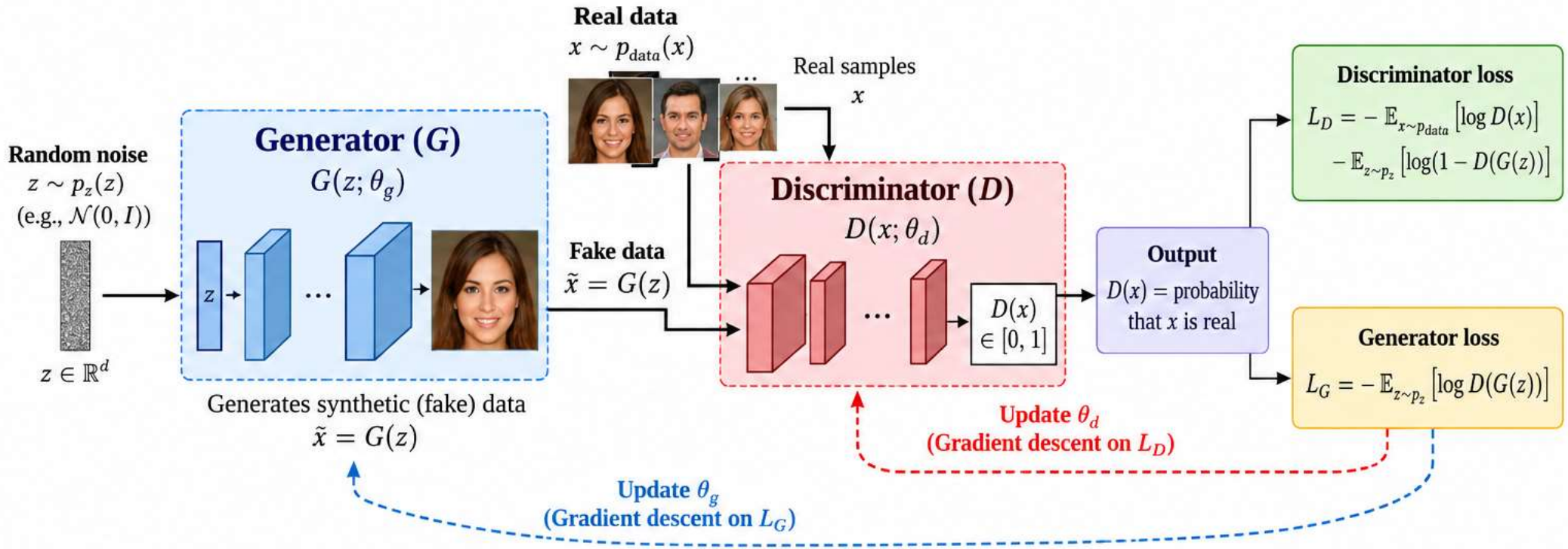

CLIP
mini batch of text-image pairs
big table
mini chair
top hat
table top
big chair
word2vec
mini big top hat chair table
0 1 1 0 1 0
1 0 0 0 1 1
0 1 0 1 0 1
Flatten Patches
Word Embeddings
Image Encoder
Text Encoder
[Mean Pooling] (round)
[Mean Pooling] (round)
[Projection]
[Projection]
Shared Embedding Space
T1 T2 T3
[Softmax]
Similarity Image→Text
Target
Cross Entropy Loss Gradients
Image→Text
Text→Image
Similarity Text→Image


1. Offline: Indexing (Create and Store Vectors)
Documents / Text Corpus
D1: "how are you"
D2: "who are you"
D3: "who am I"
...
Tokenization
how are you
who are you
who am I
...
Word Embeddings
how [1 0 1 0]
are [0 2 0 1]
you [0 0 1 0]
who [-1 1 0 0]
am [0 1 0 1]
I [0 0 1 1]
... ...
Text Embedding Encoder
Linear Layer (Wx + b)
ReLU
Mean Pooling
Text Embedding (Embedding Vector)
2/3
2/3
1/3
...
Projection (to DB dimension)
$W_p x + b_p$
Document Vector (to store)
5/3
2/3
...
VECTOR DATABASE
Vector Storage
(vectors + metadata)
ID | Vector (d-dim) | Metadata
D1 | [5/3, 2/3, ...] | {text: "how are you"}
D2 | [5/3, 2/3, ...] | {text: "who are you"}
D3 | [7/3, 2/3, ...] | {text: "who am I"}
... | ... | ...
Vector Index
(Approximate Nearest Neighbor)
HNSW / IVF / PQ / Flat
2. Online: Query and Retrieval
User Query
"who am I"
Tokenization
who am I
Word Embeddings
who → [-1 1 0 0]
am → [0 1 0 1]
I → [0 0 1 1]
Text Embedding Encoder
Linear + ReLU
Mean Pooling
Query Vector
8/3
2/3
...
Similarity Search
(Dot Product / Cosine / L2)
$score(q, v_i) = q^T v_i$
Nearest Neighbor
(argmax)
D1: 44/9
D2: 40/9
D3: 60/9 (✓)
D4: 32/9
...
Top-K Results
1. "who am I"
2. ...
3. ...

Vector Database



Generative Adversarial Network (GAN) Architecture



1. Discriminator Objective

Maximize the probability of correctly classifying real and fake samples:

$$L_D = -\mathbb{E}_{x \sim p_{data}}[\log D(x)] - \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z))]$$

$D(x) \longrightarrow 1$ for real data

$D(G(z)) \longrightarrow 0$ for generated data

2. Generator Objective

Generate samples that fool the discriminator:

$$L_G = -\mathbb{E}_{z \sim p_z}[\log D(G(z))]$$

G tries to maximize $D(G(z))$
(i.e., make generated samples look real).

3. Minimax Game

The generator and discriminator are trained in a two-player minimax game:

$$\min_G \max_D \; V(D, G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z))]$$

At the optimal point:

- $G(z) \sim p_{data}(x)$ (generated distribution matches real)
- $D(x) = \frac{1}{2}$ (discriminator cannot distinguish real from fake)

Generative Adversarial Network (GAN) — Toy Numerical Example

Random noise z

$z \in \mathbb{R}^{2\times4}$

N_1	N_2	N_3	N_4
1	1	0	1
1	0	1	−1

Generator (G)

Weight matrix $W_G \in \mathbb{R}^{3\times2}$

1	1	0
0	1	2
−1	1	0

Linear + ReLU ≈ ReLU

Fake data (generated)

$F = G(z) \in \mathbb{R}^{4\times4}$

F_1	F_2	F_3	F_4
2	1	1	0
3	2	3	1
0	0	1	2
1	2	0	1

Real data

$X \in \mathbb{R}^{4\times4}$

X_1	X_2	X_3	X_4
2	3	3	4
1	1	1	1
2	3	4	3
1	1	1	1

Discriminator (D)

Weight matrix $W_D \in \mathbb{R}^{3\times5}$

1	0	0	−1	0
0	1	1	0	0
0	0	1	−1	1

Linear + ReLU ≈ ReLU

$z = W_0\ h \in \mathbb{R}^{1\times4}$ (logits / scores)

1	1	−1	−1

Sigmoid $\sigma(\cdot)$

$D(F)$ (fake samples) $\sigma(z_F)$

0.7	0.5	0.9	0.3

$D(X)$ (real samples) $\sigma(z_X)$

0.7	0.9	0.9	1.0

1. Train Discriminator (D)

Targets:

Fake (F): $y_D = 0$

0	0	0	0

Real (X): $y_D = 1$

1	1	1	1

Loss (binary cross-entropy):

$$L_D = -\left[\frac{1}{4}\sum_{i=1}^{4}\log D(x_i) + \frac{1}{4}\sum_{i=1}^{4}\log(1 - D(G(z_i)))\right]$$

Gradients $\partial L_D/\partial\theta_D$ (update θ_D)

2. Train Generator (G)

Targets for fake samples:

$y_G = 1$ (want $D(G(z_i))$ to be real)

1	1	1	1

Loss (generator):

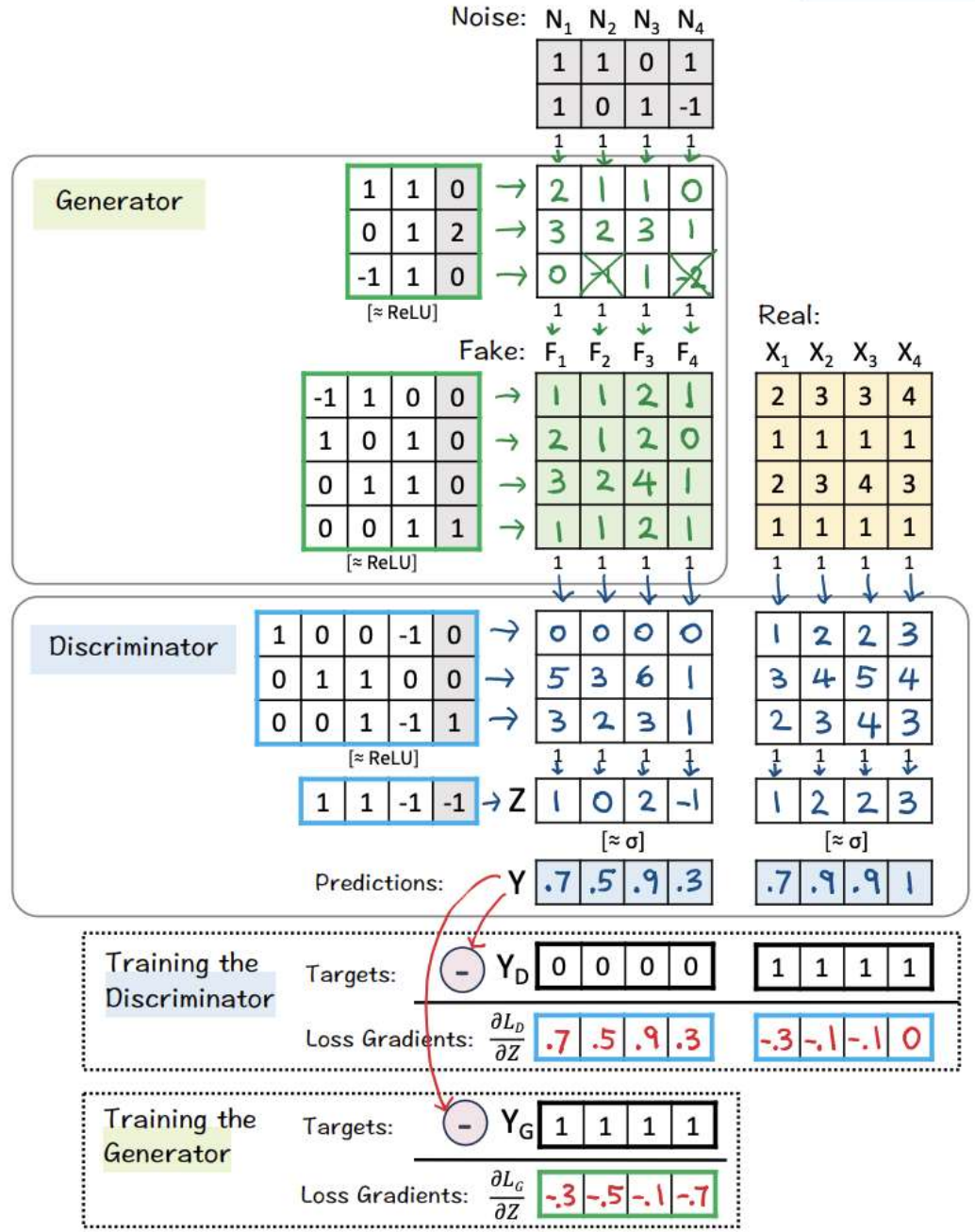
$$L_G = -\frac{1}{4}\sum_{i=1}^{4}\log D(G(z_i))$$

Gradients $\partial L_G/\partial\theta_G$ (update θ_G)

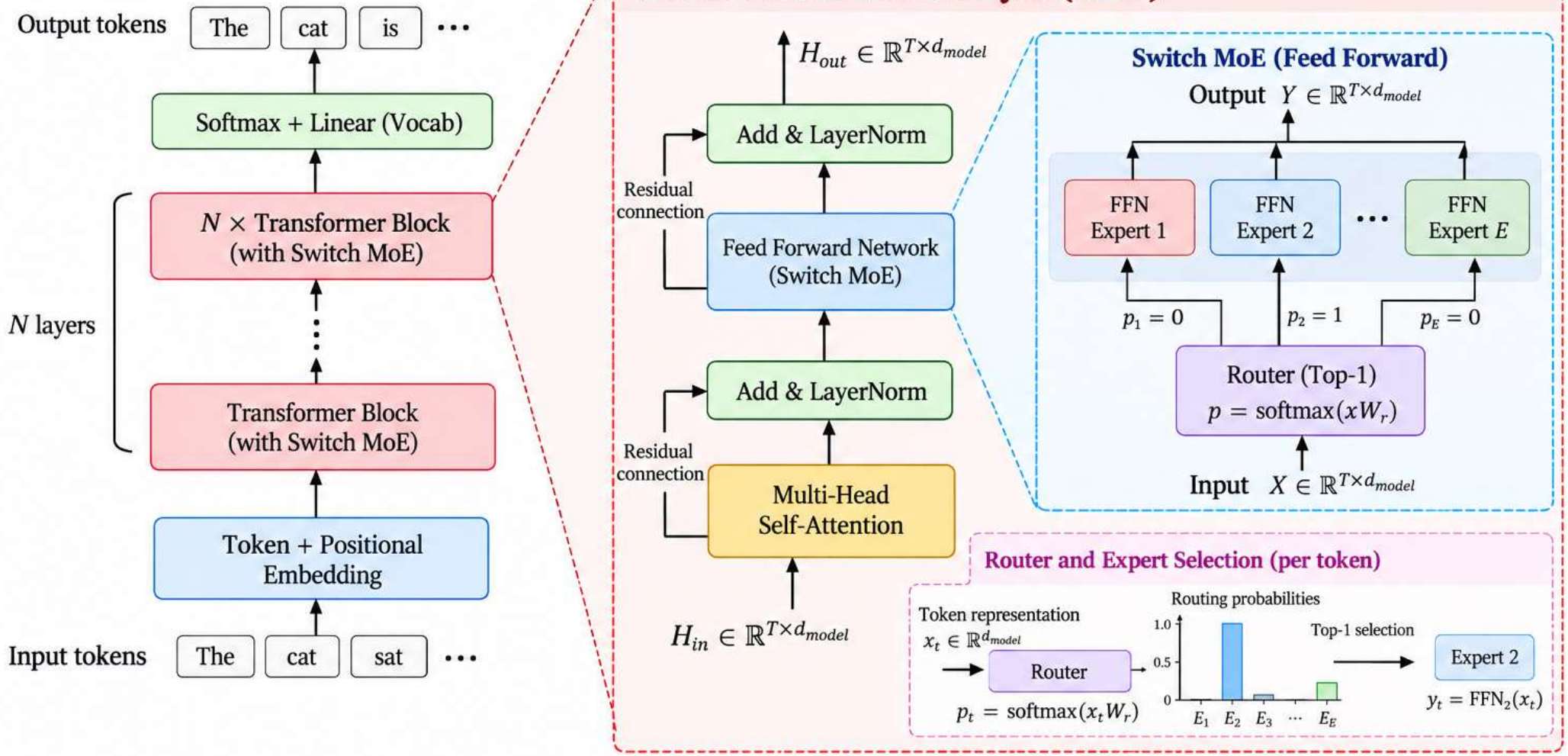
Overall objective (minimax game)

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z_i))]$$

Generative Adversarial Network (GAN)



Switch Transformer Architecture



Expert Feed Forward (standard FFN)

$$\text{FFN}_i(x) = W_{2,i}\sigma(W_{1,i}x + b_{1,i}) + b_{2,i}$$

Router (linear projection)

$$p_t = \text{softmax}(x_t W_r), \quad W_r \in \mathbb{R}^{d_{model} \times E}$$

Auxiliary Load-Balancing Loss

$$L_{\text{aux}} = \text{E} \sum_{i=1}^{E} f_i \cdot P_i$$

f_i : fraction of tokens assigned to expert i

P_i : mean routing probability of expert i

Switch Transformer Architecture
(Sparse Mixture of Experts, e.g., Gemini 1.5)

1. Input tokens (from previous block)

Hidden states $X \in \mathbb{R}^{T \times d_{model}}$

X_1	X_2	X_3	X_4	X_5
5	6	0	7	0
0	2	4	0	3
1	0	1	1	0

$T = 5$ tokens

$d_{\text{model}} = 3$ (toy example)

2. Self-Attention

Compute Q, K and attention weights

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

Q
K
V

Attention Weight Matrix A

	X_1	X_2	X_3	X_4	X_5
X_1	1	0	0	0	1
X_2	1	1	0	0	0
X_3	0	1	1	0	0
X_4	0	0	1	1	0
X_5	0	0	0	1	1

Attention weighted token representations

Z_1	Z_2	Z_3	Z_4	Z_5
11	6	7	7	5
2	6	4	3	3
1	1	2	1	1

$Z \in \mathbb{R}^{T \times d_{model}}$

3. Router (Switch)

Linear projection + softmax

$$p_t = \text{softmax}(x_t W_r)$$

Router logits (scores)

	X_1	X_2	X_3	X_5
A	1	−1	0	0
B	1	0	−2	
C	−1	1	1	

Gate values (routing probabilities)

	X_1	X_2	X_3	X_4	X_5
A	0.9	0.0	0.3	0.4	0.2
B	0.1	0.4	0.3	0.5	0.3
C	0.0	0.6	0.4	0.1	0.5

argmax (Top-1 expert per token)

Expert IDs: A | C | C | B | C

4. Expert FFNs (only selected expert runs for each token)

Expert A (FFN)

Tokens routed to A: Z_1

W_A

1	0	−1	0
0	1	0	0
0	0	−1	1

Z_1: 11, 2, 1 → FFN_A → 10, 2, 0

Expert B (FFN)

Tokens routed to B: Z_4

W_B

0	0	−1	0
1	0	0	0
1	1	−1	1

Z_4: 7, 3, 1 → FFN_B → −1, 7, 10

Expert C (FFN)

Tokens routed to C: Z_2, Z_3, Z_5

W_C

1	0	0	0
0	1	1	1
0	1	0	0

Z_2	Z_3	Z_5
6	7	5
6	4	3
1	2	1

→ FFN_C →

6	7	2
8	7	3
6	4	4

5. Combine expert outputs and form next block input

$Y \in \mathbb{R}^{T \times d_{model}}$

Y_1	Y_2	Y_3	Y_4	Y_5
10	6	7	−1	2
2	8	7	7	3
0	6	4	10	4

Add & LayerNorm

Residual connection (from attention output Z)

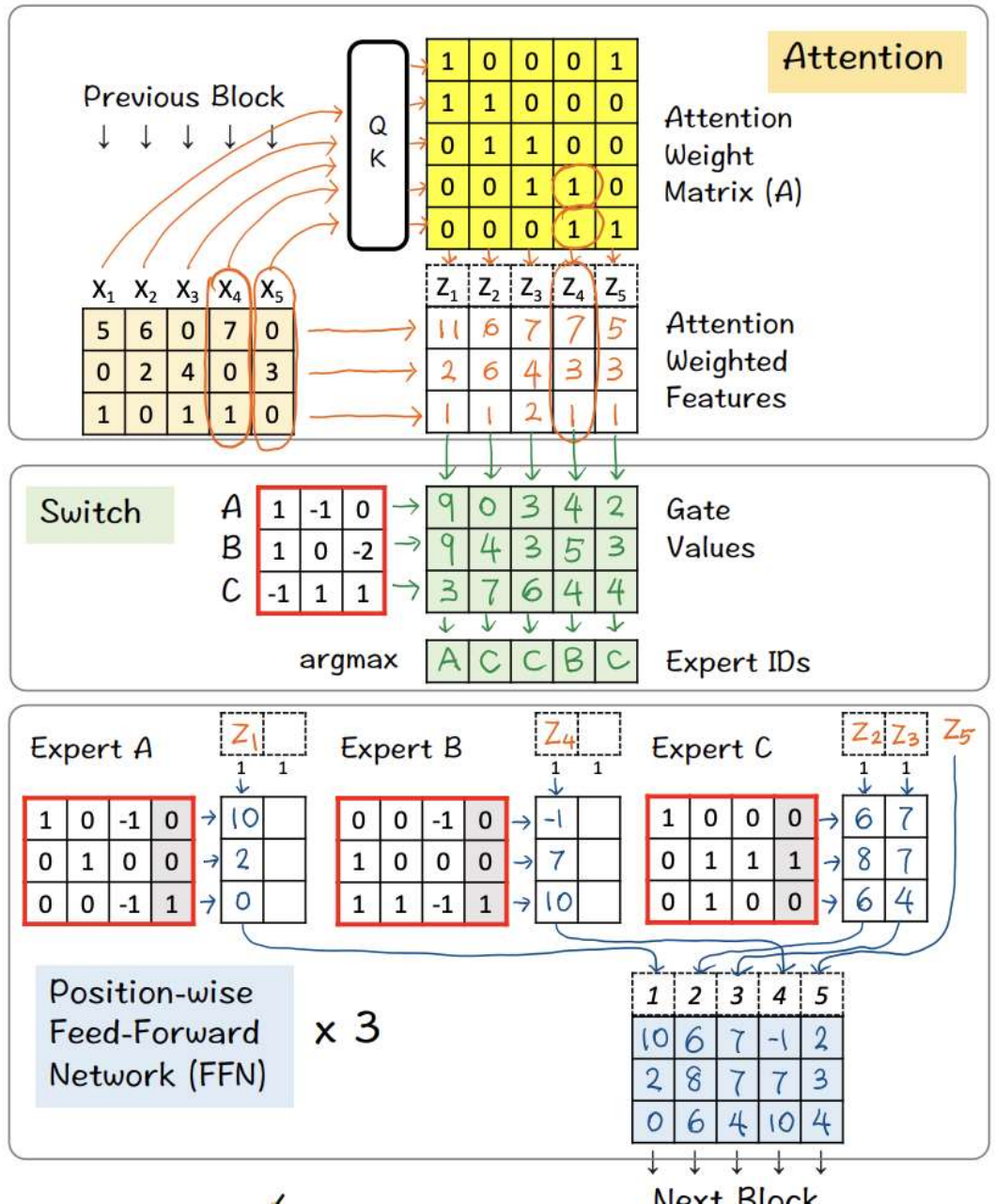
Next Transformer Block (with Switch MoE) ...

6. Key idea

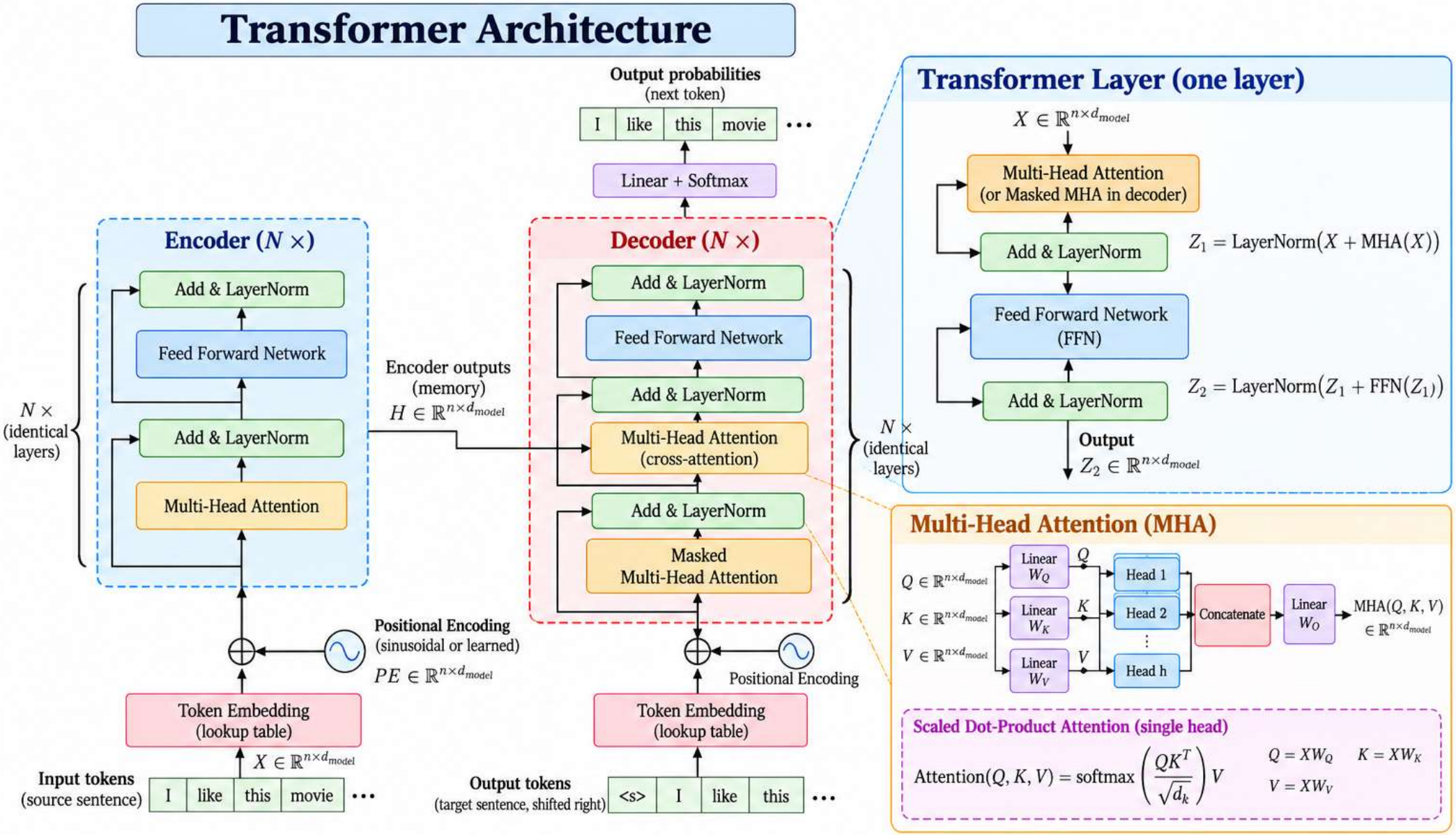
- Each token is routed to only one expert (Top-1). → sparse computation.
- Different tokens can go to different experts.
- Router learns which expert to use.
- Much more efficient than dense MoE (only $1/E$ of experts are used per token).

Switch Transformer

(Gemini 1.5's Sparse Mixture of Experts)



Transformer Architecture



Transformer Layer (one layer)

$X \in \mathbb{R}^{n \times d_{model}}$

Multi-Head Attention (or Masked MHA in decoder)

Add & LayerNorm

$Z_1 = \text{LayerNorm}(X + \text{MHA}(X))$

Feed Forward Network (FFN)

Add & LayerNorm

$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$

Output $Z_2 \in \mathbb{R}^{n \times d_{model}}$

Multi-Head Attention (MHA)

Scaled Dot-Product Attention (single head)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Position-wise Feed Forward Network (FFN)

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \qquad W_1 \in \mathbb{R}^{d_{model} \times d_{ff}} \qquad W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$$

Positional Encoding (*sinusoidal, original Transformer*)

$$\text{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \qquad \text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Transformer Block: Self-Attention + Position-wise Feed-Forward Network

1. Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Input token features (from previous block)

$X \in \mathbb{R}^{3\times5}$

X_1	X_2	X_3	X_4	X_5
5	6	0	7	0
0	2	4	0	3
1	0	1	1	0

3 features, 5 tokens (each column is a token)

Q, K (from X)

Attention weight matrix $A \in \mathbb{R}^{5\times5}$

	X_1	X_2	X_3	X_4	X_5
X_1	1	0	0	0	1
X_2	1	1	0	0	0
X_3	0	1	1	0	0
X_4	0	0	1	1	0
X_5	0	0	0	1	1

Attention-weighted features $Z = XA$ (each token attends to others)

$Z \in \mathbb{R}^{3\times5}$

Z_1	Z_2	Z_3	Z_4	Z_5
11	6	7	7	5
2	6	4	3	3
1	1	2	1	1

Residual connection (from previous block)

LayerNorm

2. Position-wise Feed-Forward Network (FFN)

(applied independently to each token)

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Input (from attention) $Z \in \mathbb{R}^{3\times5}$

Z_1	Z_2	Z_3	Z_4	Z_5
11	6	7	7	5
2	6	4	3	3
1	1	2	1	1

First linear layer $W_1 \in \mathbb{R}^{4\times3}$ (applied to each token)

1	−1	0
1	1	0
0	1	1
−1	1	0

Hidden values $(ZW_1) \in \mathbb{R}^{4\times5}$

Z_1	Z_2	Z_3	Z_4	Z_5
10	1	4	5	3
13	12	11	10	8
4	8	7	5	5
−9	1	−1	−4	−1

ReLU (h = max(0, x))

After ReLU $H \in \mathbb{R}^{4\times5}$

Z_1	Z_2	Z_3	Z_4	Z_5
10	1	4	5	3
13	12	11	10	8
4	8	7	5	5
0	1	0	0	0

Second linear layer $W_2 \in \mathbb{R}^{3\times4}$ (applied to each token)

1	0	0	−1	0
0	1	1	0	0
0	0	1	−1	1

Output of FFN $Y \in \mathbb{R}^{3\times5}$

Y_1	Y_2	Y_3	Y_4	Y_5
10	6	7	−1	2
2	8	7	7	3
0	6	4	10	4

Residual connection (from attention output)

LayerNorm

Output to next Transformer block

Transformer

